

Customer Churn Prediction System

End-to-End ML System with CRISP-DM & MLOps (MLflow)

Hector Alejandro Vargas Gutierrez

June 18, 2026

Modeling Strategy and Hyperparameter Optimization

Optimization Framework

Optuna's Tree-structured Parzen Estimator (TPE) was used to maximize **Average Precision (PR-AUC)** through a Bayesian search of **5,000 trials**.

- End-to-end pipeline including:
 - Dynamic feature engineering
 - Scaling strategy selection (Standard, MinMax, Robust)
 - Encoding strategy selection
 - Model selection (Random Forest vs XGBoost)
- Evaluation performed using **5-fold Stratified Cross Validation**.
- Optuna's **MedianPruner** stopped poorly performing trials early, reducing computational cost.
- Optimization target: **PR-AUC**, chosen because churn prediction is an imbalanced classification problem.

Experiment Tracking with MLflow

Reproducible Optimization Workflow

MLflow was integrated with Optuna to automatically track all hyperparameter optimization experiments.

- SQLite backend used for persistent experiment storage.
- Parent-child run structure:
 - Parent run stores final model results.
 - Child runs store individual Optuna trials.
- Automatic logging of:
 - Hyperparameters
 - Cross-validation scores
 - Test-set metrics
 - Feature schema
 - Serialized production pipeline
- Ensures full reproducibility and auditability of model development.

Best Model Performance

Metric	Value
Cross-Validation PR-AUC	0.701
Test PR-AUC	0.736
Accuracy	0.865
Precision	0.781
Recall	0.598
F1 Score	0.666
ROC-AUC	0.876

Best Configuration

XGBoost with Robust Scaling, 1,193 trees, learning rate 0.0267, and class imbalance correction using `scale_pos_weight = 1.67`.

Hyperparameter Search Insights

Best Configuration Identified by Optuna

- Model: XGBoost
 - Scaler: RobustScaler
 - Trees: 1,193
 - Learning Rate: 0.0267
 - Subsample: 0.94
 - Column Sample Rate: 0.51
 - Gamma: 3.33
 - Scale Positive Weight: 1.67
-
- XGBoost consistently outperformed Random Forest across the search space.
 - Robust scaling produced the strongest results.
 - Moderate class weighting improved minority-class detection.

Key Drivers of Churn

- Low usage frequency
- Short customer tenure
- Payment delays
- Weak engagement score

Model Explainability

- Feature importance analysis
- SHAP values for individual predictions
- Business interpretation of churn drivers

System Architecture

- Data ingestion pipeline
- Feature engineering pipeline
- ML model training (MLflow tracking)
- Model registry
- API inference service

Churn Prediction Workflow

- Customer data input
- Feature transformation
- Model inference
- Churn probability output
- Business action (retention campaign)

- REST API (FastAPI / Flask)
- Docker containerization
- Model served from MLflow registry

Production Monitoring

- API latency and error tracking
- Data drift detection
- Prediction distribution monitoring
- Model performance degradation alerts

End-to-End MLOps Pipeline

Data → Features → Training → MLflow → Deployment → Monitoring → Retraining

Challenges

- Class imbalance in churn data
- Feature leakage risks
- Model interpretability vs performance tradeoff

Future Improvements

- Real-time churn scoring
- Automated retraining pipeline
- A/B testing retention strategies
- Improved feature store integration

Conclusion

- Built end-to-end churn prediction system
- Achieved strong predictive performance ($F1 = XX\%$)
- Implemented full MLOps lifecycle with MLflow
- System ready for production decision support

Thank You